Guía Técnica de Configuración del Entorno de Computación Científica

Maestría en Computación y Matemática — Geometría Lineal

David Duarte

1 Introducción y Requerimientos del Sistema

El curso de **Geometría Lineal y Computacional** exige el desarrollo de algoritmos numéricos, manipulaciones simbólicas y representaciones gráficas tridimensionales continuas y discretas. Para garantizar reproducibilidad en las evaluaciones y laboratorios computacionales, cada estudiante debe configurar un entorno de trabajo unificado basado en Python (y opcionalmente Julia), integrando librerías avanzadas de álgebra matricial, procesamiento de nubes de puntos y animación espacial.

1.1 Especificaciones Mínimas Recomendadas

- **Sistema Operativo:** GNU/Linux (Ubuntu 22.04 LTS o superior recomendado), macOS (Apple Silicon o Intel) o Windows 10/11 con *Windows Subsystem for Linux 2 (WSL2)*.
- **Memoria RAM:** 8 GB mínimo (16 GB recomendado para el cálculo de operadores sobre mallas densas).
- **Procesador y Gráficos:** CPU multinúcleo x86_64 o ARM64; tarjeta gráfica con soporte OpenGL 3.3 o superior (necesario para el renderizado interactivo en Open3D y Blender).

2 Gestor de Entornos: Miniforge y Conda

Para evitar conflictos entre las librerías del sistema y paquetes compilados en C/C++ (como Open3D o SciPy), se utilizará **Miniforge** (distribución ligera de Conda enfocada en el canal comunitario `conda-forge`).

2.1 Paso 1: Instalación de Miniforge

Descargue e instale Miniforge según su sistema operativo:

```
# En sistemas GNU/Linux y macOS:
curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/
    download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
```

Siga las instrucciones en la terminal, acepte la licencia y autorice la inicialización de Conda (`conda init`). Reinicie su terminal al finalizar.

2.2 Paso 2: Creación del Entorno Virtual Dedicado

Se creará un entorno virtual aislado denominado `geomlineal` con Python 3.10 (versión con máxima compatibilidad entre Open3D, Trimesh y las APIs de Blender):

```
conda create -n geomlineal python=3.10 -y
conda activate geomlineal
```

3 Instalación del Ecosistema de Computación Científica

3.1 Paso 3: Núcleo Matemático y Servidor JupyterLab

Instale las herramientas fundamentales de álgebra lineal numérica, integración simbólica y el entorno de cuadernos interactivos:

```
conda install -c conda-forge numpy scipy matplotlib sympy jupyterlab ipympl
    ipywidgets -y
```

- **NumPy y SciPy:** Computación matricial vectorizada, descomposición espectral, SVD e integración numérica de ecuaciones diferenciales ordinarias (`solve_ivp`) para geodésicas.
- **SymPy:** Deducción formal exacta de coeficientes métricos, curvaturas y tensores.
- **ipympl:** Habilita rotación, zoom y visualización interactiva tridimensional dentro de las celdas de JupyterLab.

3.2 Paso 4: Librerías de Geometría Computacional y Mallas 3D

Instale las librerías especializadas en nubes de puntos y geometría discreta:

```
pip install open3d trimesh rtree shapely
```

- **Open3D:** Registro rígido por ICP (Iterative Closest Point), estimación de normales en superficies y reconstrucción por mínimos cuadrados móviles (MLS).
- **Trimesh:** Estructuras de datos de semi-aristas (*half-edge*), cálculo discreto del operador de Laplace-Beltrami, curvatura gaussiana y comprobación de cerradura topológica (*watertight*).

4 Configuración de Blender y su API en Python (`bpy`)

Para la visualización cinemática de manipuladores robóticos y la renderización de variedades afines y campos vectoriales continuos, existen dos metodologías de integración con Blender:

4.1 Método A: Automatización Externa por Línea de Comandos (Recomendado)

Este método es el más estable y no altera el entorno virtual:

1. Descargue e instale **Blender** (versión 3.6 LTS o 4.x) desde el sitio web oficial (`https://www.blender.org`).

2. Asegúrese de que el binario ejecutable esté en la variable de entorno `PATH`.
3. Ejecute cualquier script desarrollado en el curso en modo desatendido (*background*) mediante el comando:

```
blender --background --python script_afines.py
```

4. Para inyectar paquetes como `scipy` o `trimesh` dentro del intérprete Python embebido de Blender, use la terminal:

```
# En Linux (ajustar ruta segun instalacion):
/opt/blender/4.2/python/bin/python3.11 -m pip install scipy trimesh
```

4.2 Método B: Paquete `bpy` Independiente vía Pip

Si desea importar Blender directamente dentro de un script estándar de Python (`import bpy`), instale la compilación precompilada (requiere exactamente la versión de Python que demanda la compilación de la rueda):

```
pip install bpy==4.0.0
```

5 Entorno Julia (Opcional para Cálculo Diferencial de Alto Rendimiento)

Para estudiantes orientados al análisis tensorial simbólico-numérico y la resolución acelerada de geodésicas:

5.1 Paso 5: Instalación de Julia y Paquetes Esenciales

1. Instale Julia mediante el gestor oficial `juliaup`:

```
curl -fsSL https://install.julialang.org | sh
```

2. Abra el REPL de Julia y presione la tecla `]` para acceder al gestor de paquetes (`Pkg`):

```
(@v1.10) pkg> activate @geomlineal
(@geomlineal) pkg> add LinearAlgebra ForwardDiff DifferentialEquations
   Meshes IJulia
```

3. Con `IJulia`, el kernel de Julia quedará automáticamente disponible dentro del mismo servidor JupyterLab configurado en Python.

6 Script de Verificación del Entorno

Para certificar que el sistema está completamente configurado y libre de errores antes de comenzar los laboratorios computacionales, cree un archivo llamado `verificar_entorno.py` y ejecútelo en su terminal con el entorno activado:

```
import sys
import numpy as np
import scipy
from scipy.linalg import svd
import trimesh
```

```
print(f"[OK] Python version: {sys.version.split()[0]}")
print(f"[OK] NumPy version:  {np.__version__}")
print(f"[OK] SciPy version:  {scipy.__version__}")
print(f"[OK] Trimesh version:{trimesh.__version__}")

# Prueba de Descomposicion en Valores Singulares (SVD)
A = np.array([[2.0, 0.0, 1.0], [-1.0, 2.0, 1.0]])
U, s, Vt = svd(A)
assert np.allclose(A, U[:, :2] @ np.diag(s) @ Vt), "Error en SVD"
print("[OK] Calculo matricial SVD validado correctamente.")

# Prueba de Geometria Discreta
caja = trimesh.creation.box(extents=[1.0, 1.0, 1.0])
print(f"[OK] Malla de prueba generada: {len(caja.vertices)} vertices, "
      f"{len(caja.faces)} caras. Cerrada: {caja.is_watertight}")

# Prueba de Open3D
try:
    import open3d as o3d
    print(f"[OK] Open3D version: {o3d.__version__}")
except ImportError:
    print("[ADVERTENCIA] Open3D no esta instalado o requiere drivers OpenGL
        .")

# Prueba de Blender BPY
try:
    import bpy
    print(f"[OK] Blender BPY disponible (Version {bpy.app.version_string})"
        )
except ImportError:
    print("[INFO] BPY standalone no cargado. Se utilizara ejecucion externa
        .")

print("\n>>> El entorno de computacion cientifica esta listo para su uso.")
```

Ejecución del test:

```
python verificar_entorno.py
```

7 Solución de Problemas Frecuentes (Troubleshooting)

- **Fallo de visualización en JupyterLab (`ipympl`):** Si los gráficos 3D no responden a la rotación del cursor, verifique haber incluido la directiva mágica `%matplotlib widget` al inicio de su cuaderno.
- **Errores de OpenGL en Linux / WSL2:** Si Open3D falla al abrir ventanas de visualización, ejecute:

  ```
  sudo apt-get install libgl1-mesa-glx libglib2.0-0
  ```

- **Conflictos con paquetes del sistema:** Asegúrese de no mezclar instalaciones globales de `sudo pip` con las del entorno Conda. Trabaje siempre bajo el prompt `(geomlineal)`.